



Plataforma Web de Streaming de Conteúdos Multimédia

Programação Web — CTPSI — IPS · Ano letivo 2025/2026

Alunos Rodrigo · N.º 2025126234

Zivile Sink · N.º 2025166759

E-mail info@evolsolutions.pt

Data Junho 2026

Instituto Politécnico de Setúbal — Escola Superior de Tecnologia de Setúbal

Índice

Arquitetura do sistema

- Visão geral
- Estrutura de ficheiros
- Fluxo de um pedido

Entidades e API REST

- Modelo de dados (diagrama)
- Tabelas da base de dados
- Rotas por entidade

Autenticação e sessões

Integração frontend-backend (AJAX)

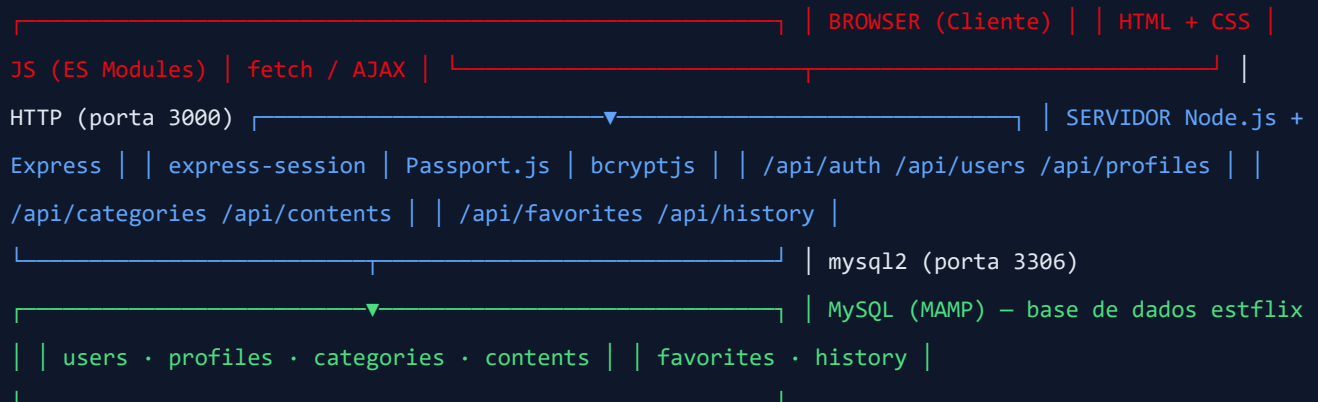
Descrição das opções tomadas

Limitações técnicas e trabalho futuro

1. Arquitetura do Sistema

1.1 Visão geral

A ESTFlix segue uma arquitetura cliente–servidor de três camadas: o browser executa o frontend (HTML + CSS + JavaScript com ES Modules), comunica via HTTP com um servidor Node.js/Express que expõe uma API REST, e este por sua vez persiste os dados num servidor MySQL gerido pelo MAMP.



1.2 Estrutura de ficheiros

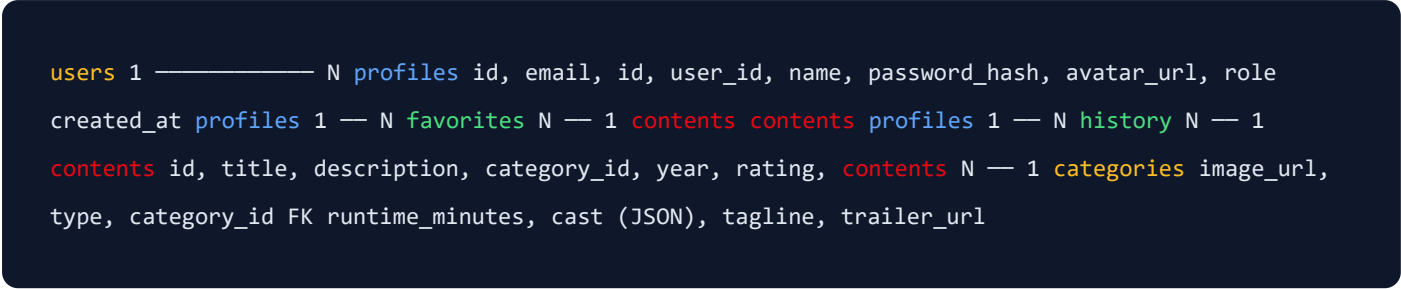
ESTFlix/	
├─ server.js	Ponto de entrada – Express app, middlewares, rotas
├─ db.js	Pool de ligações MySQL (mysql2/promise)
├─ schema.sql	Script DDL de criação da base de dados
├─ seed.sql	Inserts SQL puros (alternativa ao seed.js)
├─ seed.js	Popular a BD com dados iniciais via Node.js
├─ package.json	Dependências npm
├─ routes/	
│ └─ auth.js	Autenticação: /api/auth/me login logout register profile
│ └─ users.js	CRUD /api/users
│ └─ profiles.js	CRUD /api/profiles
│ └─ categories.js	CRUD /api/categories
│ └─ contents.js	CRUD /api/contents
│ └─ favorites.js	/api/favorites
│ └─ history.js	/api/history + /api/history/recommendations
├─ js/	
│ └─ services/	
│ │ └─ apiService.js	Wrapper fetch (api.get/post/put/del)
│ └─ pages/	Uma página JS por HTML (loginPage, homePage, ...)
│ └─ models/	Classes de domínio (Content, Category, Profile, User)
│ └─ ui/	
│ │ └─ dom.js	Funções el() e clear() para construção do DOM
├─ css/	
│ └─ styles.css	Ponto de entrada CSS (apenas @import)
│ └─ variables.css	Variáveis CSS (cores, fontes, espaçamentos)
│ └─ base.css	Reset e estilos base do documento
│ └─ typography.css	Headings, parágrafos, links
│ └─ navbar.css	Barra de navegação
│ └─ buttons.css	Botões e variantes
│ └─ forms.css	Inputs, labels, formulários
│ └─ layout.css	Grid, container, hero, secções
│ └─ poster.css	Cards/posters de conteúdo
│ └─ profile.css	Cartões de perfil
│ └─ admin.css	Painel de administração
│ └─ detail.css	Página de detalhe de conteúdo
│ └─ responsive.css	Media queries (max-width: 820px e 500px)
└─ docs/	Manuais (este ficheiro + manual_utilizador.html)

1.3 Fluxo de um pedido

1. O browser carrega `home.html` do servidor Express (static files).
2. `homePage.js` executa `api.get('/contents')` via `apiService.js`.
3. O `fetch` envia `GET /api/contents` com `credentials: 'same-origin'`.
4. Express encaminha para `routes/contents.js`, que consulta o MySQL via `db.js`.
5. A rota devolve JSON; o `apiService` lança excepção se `!res.ok`.
6. A página usa `el()` para construir os cards no DOM (sem `innerHTML`).

2. Entidades e API REST

2.1 Modelo de dados



2.2 Tabelas da base de dados

Tabela	Colunas principais	Observações
users	id, email, password_hash, created_at	password_hash com bcrypt (factor 10)
profiles	id, user_id, name, avatar_url, role	role: ENUM('USER','ADMIN')
categories	id, name UNIQUE	—
contents	id, title UNIQUE, description, category_id, year, rating, image_url, type, runtime_minutes, cast, tagline, trailer_url	cast é coluna JSON; type: ENUM('movie','series')
favorites	profile_id, content_id, created_at	PK composta; ON DELETE CASCADE
history	profile_id, content_id, watched_at, times	PK composta; ON DUPLICATE KEY UPDATE times+ 1

2.3 Rotas por entidade

Autenticação — /api/auth

Método	Rota	Descrição	Auth
GET	/api/auth/me	Devolve utilizador, activeProfileId e perfil da sessão	—
POST	/api/auth/register	Cria utilizador + perfil default; faz login automático	—
POST	/api/auth/login	Passport LocalStrategy; define activeProfileId na sessão	—
POST	/api/auth/logout	req.logout() + session.destroy()	✓
PUT	/api/auth/profile	Altera activeProfileId na sessão	✓

Utilizadores — /api/users

Método	Rota	Descrição	Auth
GET	/api/users	Lista todos os utilizadores (id, email, created_at)	✓
GET	/api/users/:id	Detalhe de um utilizador	✓
POST	/api/users	Cria utilizador + perfil default	—
PUT	/api/users/:id	Atualiza email/password (conta própria)	✓
DELETE	/api/users/:id	Elimina conta (faz logout)	✓

Perfis — /api/profiles

Método	Rota	Descrição	Auth
GET	/api/profiles	Lista perfis do utilizador autenticado	✓
GET	/api/profiles/:id	Detalhe de um perfil (verificação de propriedade)	✓
POST	/api/profiles	Cria novo perfil para o utilizador autenticado	✓
PUT	/api/profiles/:id	Atualiza nome/avatar	✓
DELETE	/api/profiles/:id	Elimina perfil (e favoritos/history em cascade)	✓

Categorias — /api/categories

Método	Rota	Descrição	Auth
GET	/api/categories	Lista todas as categorias (público)	—
GET	/api/categories/:id	Detalhe de categoria (público)	—
POST	/api/categories	Cria categoria; valida nome único (LOWER)	✓
PUT	/api/categories/:id	Atualiza nome; valida unicidade	✓
DELETE	/api/categories/:id	Elimina; devolve 409 se tiver conteúdos associados	✓

Conteúdos — /api/contents

Método	Rota	Descrição	Auth
GET	/api/contents	Lista; suporta ?type=, ?category=, ?q= (LIKE) (público)	—
GET	/api/contents/:id	Detalhe de um conteúdo (público)	—
POST	/api/contents	Cria conteúdo; cast serializado como JSON	✓
PUT	/api/contents/:id	Atualiza todos os campos	✓
DELETE	/api/contents/:id	Elimina (favoritos/history em cascade)	✓

Favoritos — /api/favorites

Método	Rota	Descrição	Auth
GET	/api/favorites	Lista favoritos; ?profileId= ou sessão	✓
GET	/api/favorites/:profileId/:contentId	Verifica se favorito existe	✓
POST	/api/favorites	Adiciona favorito (INSERT IGNORE — idempotente)	✓
PUT	/api/favorites/:profileId/:contentId	Atualiza metadados	✓
DELETE	/api/favorites/:profileId/:contentId	Remove favorito	✓

Histórico — /api/history

Método	Rota	Descrição	Auth
GET	/api/history/recommendations	Recomendações baseadas nos géneros mais vistos	✓
GET	/api/history	Histórico de visualizações; ?profileId=	✓
POST	/api/history	Regista visualização; ON DUPLICATE KEY UPDATE times+1	✓
PUT	/api/history/:profileId/:contentId	Atualiza entrada manualmente	✓
DELETE	/api/history/:profileId/:contentId	Remove entrada do histórico	✓

3. Autenticação e Sessões

A autenticação é gerida pelo módulo **Passport.js** com a estratégia *LocalStrategy*.

Fluxo de login

1. O frontend envia `POST /api/auth/login` com `{ email, password }`.
2. A *LocalStrategy* procura o utilizador por email na tabela `users`.
3. Compara a password com o hash armazenado via `bcryptjs.compare()`.
4. Em caso de sucesso, `req.login()` serializa o `id` do utilizador na sessão.
5. O `activeProfileId` (primeiro perfil do utilizador) é armazenado em `req.session.activeProfileId`.
6. As sessões são mantidas por 7 dias com cookie `httpOnly`.

Hashing de passwords

Todas as passwords são armazenadas como hash bcrypt com fator de custo 10 (`bcryptjs.hash(password, 10)`). A comparação é feita com `bcryptjs.compare()`, nunca comparando plaintext.

Middleware `requireAuth`

Presente em todas as rotas que requerem autenticação. Verifica `req.user` (preenchido pelo Passport) e devolve `401` se ausente.

4. Integração Frontend–Backend (AJAX)

Toda a comunicação entre as páginas e a API é feita através do módulo `js/services/apiService.js`, que encapsula o `fetch` nativo.

```
// apiService.js (simplificado)
const BASE = '/api';

async function request(method, path, body = null) {
  const options = {
    method,
    headers: { 'Content-Type': 'application/json' },
    credentials: 'same-origin' // envia o cookie de sessão
  };
  if (body !== null) options.body = JSON.stringify(body);

  const res = await fetch(BASE + path, options);
  if (res.status === 204) return null;
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw Object.assign(new Error(data.error || res.statusText), { status: res.status });
  return data;
}

export const api = {
  get: (path) => request('GET', path),
  post: (path, body) => request('POST', path, body),
  put: (path, body) => request('PUT', path, body),
  del: (path) => request('DELETE', path)
};
```

Cada página JS importa `api` e usa `async/await` para carregar os dados necessários, construindo o DOM com a função utilitária `el()` (sem `innerHTML`).

A migração da Fase 1 (localStorage) para a Fase 2 (API) foi feita página a página, substituindo as chamadas a `StorageService` e `AuthService` por chamadas ao `apiService`.

5. Descrição das Opções Tomadas

mysql2/promise em vez de mysql

Utilizou-se o driver `mysql2` com a API de Promises nativa em vez do callback-based `mysql`, permitindo usar `async/await` em todas as rotas sem necessidade de utilitários de promisifyação. O pool de ligações garante reutilização eficiente das conexões.

ON DUPLICATE KEY UPDATE no histórico

Em vez de verificar se existe uma entrada no histórico antes de inserir (dois queries), usa-se um único `INSERT ... ON DUPLICATE KEY UPDATE times = times + 1, watched_at = NOW()`. Isto é atômico e mais eficiente, evitando race conditions.

INSERT IGNORE nos favoritos

O `INSERT IGNORE` torna a adição de um favorito idempotente: se o registo já existir (PK composta), o MySQL ignora o erro em silêncio. O cliente pode chamar o endpoint múltiplas vezes sem efeitos secundários.

Coluna JSON para o elenco (cast)

O elenco de um conteúdo é um array de strings. Armazenou-se numa coluna `JSON` nativa do MySQL 8, que garante validação de formato e permite parsing automático pelo driver `mysql2`.

Rota /recommendations antes de /:profileId

No ficheiro `routes/history.js`, a rota `GET /recommendations` é declarada **antes** de `GET /` e `DELETE /:profileId/:contentId`. Se fosse declarada depois de um padrão paramétrico, o Express capturaria a string `"recommendations"` como valor do parâmetro, causando um erro de base de dados.

snake_case → camelCase via mapRow()

O MySQL devolve colunas em `snake_case` (ex: `category_id`). Para manter consistência com as convenções JavaScript, cada rota possui uma função `mapRow()` que converte os nomes antes de serializar para JSON. O frontend nunca lida com `snake_case`.

activeProfileId na sessão (não no objeto user)

O perfil ativo é um estado de sessão transitório (pode ser alterado sem novo login). Armazena-se em `req.session.activeProfileId` em vez de no objeto Passport para não obrigar a re-serialização do utilizador. A rota `PUT /api/auth/profile` atualiza este valor.

CSS modular com @import

Em vez de um único ficheiro `styles.css` monolítico, os estilos estão divididos em 12 módulos temáticos (variáveis, base, navbar, botões, formulários, etc.), todos importados por um ficheiro de entrada `styles.css` via `@import`. Isto torna cada área do design independente, facilita a localização e modificação de estilos específicos e evita conflitos de nomes em equipas. A ordem dos imports é relevante: `responsive.css` é sempre o último, garantindo que os media queries sobrepõem as regras base.

seed.sql como alternativa ao seed.js

Para facilitar a avaliação sem obrigar à execução de `node seed.js`, foi criado um ficheiro `seed.sql` com inserts SQL puros. Os hashes bcrypt das passwords de demonstração estão pré-gerados e hardcoded no ficheiro. As foreign keys são resolvidas via subqueries (`SELECT id FROM categories WHERE name = 'Action'`), tornando os inserts independentes de IDs auto-incrementados. O ficheiro usa `INSERT IGNORE`, sendo idempotente (pode ser executado mais do que uma vez sem erro).

Servidor estático com fallback SPA

O Express serve os ficheiros estáticos da pasta `ESTFlux/` com `express.static()`. Pedidos a caminhos não-API que não correspondam a ficheiros existentes são redireccionados para `login.html`, simulando o comportamento de uma Single Page Application.

6. Limitações Técnicas e Trabalho Futuro

Limitações atuais

- **HTTPS ausente:** a aplicação corre em HTTP. Em produção seria necessário TLS/SSL, nomeadamente para proteger os cookies de sessão.
- **Sem rate limiting:** não existe proteção contra ataques de força bruta no endpoint de login.
- **Sem upload de imagens:** as imagens dos conteúdos são URLs externas (TMDB). Não é possível fazer upload local.
- **Sem refresh de sessão:** a sessão expira ao fim de 7 dias sem aviso ao utilizador.
- **Recomendações simples:** o algoritmo baseia-se apenas no género mais visto, sem pesos de preferência ou filtragem colaborativa.
- **Sem paginação:** a listagem de conteúdos devolve todos os registos de uma vez, o que pode degradar o desempenho com catálogos grandes.
- **Credenciais no código:** a password da base de dados está hardcoded no `db.js`. Em produção, deve ser movida para variáveis de ambiente (`.env`).

Ideias para desenvolvimento futuro

- Implementar paginação e ordenação nas rotas GET de conteúdos.
- Adicionar sistema de avaliação (rating) por utilizador em vez de rating fixo.
- Recomendações com filtragem colaborativa (utilizadores com histórico semelhante).
- Upload de imagens de poster e avatares de perfil.
- Notificações em tempo real com WebSockets (ex: novo conteúdo adicionado).
- Migrar para variáveis de ambiente com suporte a `.env` via `dotenv`.
- Testes automatizados com Jest para as rotas da API.
- Deploy em nuvem (Railway, Render, ou VPS) com base de dados remota.